

Построчный разбор кода лабораторной работы №3

Каждая строка каждого файла — с номером и пояснением

Содержание

1. config.py — глобальные параметры	2
2. generators.py — генераторы данных	3
2.1. Строки 1–2: Импорты	3
2.2. Строки 4–11: random_polynomial()	3
2.3. Строки 13–16: trig_function()	3
2.4. Строки 18–27: generate_noise()	4
2.5. Строки 28–33: generate_regression_sample()	4
2.6. Строки 35–50: make_circles()	4
2.7. Строки 53–66: make_xor()	5
2.8. Строки 69–86: make_blobs()	6
2.9. Строки 88–103: make_spiral()	6
2.10. Строки 105–110: CLS_DATASETS	7
2.11. Строки 112–116: train_test_split()	7
2.12. Строки 118–121: standardize()	8
2.13. Строки 123–126: standardize_targets()	8
3. mlp.py — многослойный перцептрон	9
3.1. Строки 1–9: Импорты и глобальные настройки	9
3.2. Строки 11–15: Словарь активаций	9
3.3. Строки 17–18: to_tensor()	9
3.4. Строки 20–31: build_mlp()	10
3.5. Строки 33–63: train_model() — обучение с early stopping	10
3.6. Строки 65–86: train_no_early_stop()	11
3.7. Строки 88–92: predict_regression()	12
3.8. Строки 94–98: predict_classes()	13
3.9. Строки 100–101: mse_np()	13
3.10. Строки 104–115: classification_metrics()	13
4. block1_main.py — регрессионные эксперименты	15
4.1. Строки 1–9: Импорты	15
4.2. Строки 11–28: Создание данных и константы	15
4.3. Строки 30–74: run_regression_experiments()	16
4.4. Строки 76–90: print_pivot()	18
4.5. Строки 92–127: plot_best_regression()	18
4.6. Строки 129–194: run_overfit_demo()	20
4.7. Строки 196–229: main()	22
5. block2_main.py — классификационные эксперименты	24
5.1. Строки 1–10: Импорты	24
5.2. Строки 12–24: load_cls_datasets()	24
5.3. Строки 26–68: run_classification_experiments()	25
5.4. Строки 70–84: print_cls_pivot()	26
5.5. Строки 86–133: plot_decision_boundary()	27
5.6. Строки 136–166: estimate_min_train_size()	28
5.7. Строки 171–173: kfold_indices()	30
5.8. Строки 176–209: cv_grid_search()	30
5.9. Строки 214–304: main()	31

1. config.py — глобальные параметры

Файл содержит **18 строк**. Все гиперпараметры задаются через переменные окружения с дефолтными значениями, чтобы можно было менять настройки без правки кода.

Строка	Код	Пояснение
1	<code>import os</code>	Импортируем модуль <code>os</code> стандартной библиотеки — нужен для чтения переменных окружения через <code>os.environ.get</code> .
2	(пустая строка)	Визуальное разделение: отделяем импорт от объявлений констант.
3	<code>SEED = 42</code>	Глобальный <code>seed</code> для воспроизводимости (нр. <code>random</code> , <code>torch</code>). Число 42 — произвольное, но зато каждый запуск одинаков.
4	(пустая строка)	Пустая строка-разделитель между блоками параметров.
5	<code>REG_N = int(os.environ.get('SMGM03_REG_N', '280'))</code>	Число объектов выборки для регрессии . Берётся из <code>SMGM03_REG_N</code> , если задана; иначе 280. <code>int(...)</code> приводит строку к целому.
6	<code>REG_EPS0 = float(os.environ.get(...))</code>	Полу-ширина равномерного шума $\epsilon_0 = 0.18$ для регрессии.
7	<code>REG_EPOCHS = int(os.environ.get(...))</code>	Количество эпох обучения регрессионных моделей (по умолчанию 350).
8	<code>REG_LR = float(os.environ.get(...))</code>	Learning rate для Adam-оптимизатора в задачах регрессии (0.02).
9	(пустая строка)	Разделитель между регрессионным и классификационным блоками.
10	<code>CLS_N = int(os.environ.get('SMGM03_CLS_N', '500'))</code>	Число объектов для классификационных выборок (500).
11	<code>CLS_NOISE = float(os.environ.get(...))</code>	Уровень шума в классификационных данных (0.08).
12	<code>CLS_EPOCHS = int(os.environ.get(...))</code>	Кол-во эпох классификации (250 по умолчанию; в экспериментах мы передали 500).
13	<code>CLS_LR = float(os.environ.get(...))</code>	Learning rate классификации (0.03).
14	(пустая строка)	Разделитель.
15	<code>CV_K = int(os.environ.get('SMGM03_CV_K', '5'))</code>	Число фолдов для <i>k</i> -fold кросс-валидации (5).
16	(пустая строка)	Разделитель.
17	<code>DPI = 130</code>	Разрешение сохраняемых PNG-изображений (точек на дюйм).
18	<code>AXIS_LIMIT = 6.0</code>	Граница осей для визуализации (используется при отрисовке Grid в GUI).

2. generators.py — генераторы данных

Файл содержит 130 строк. Реализует генерацию регрессионных и классификационных выборок, а также `train_test_split` и стандартизацию.

2.1. Строки 1–2: Импорты

Строка	Код	Пояснение
1	<code>import numpy as np</code>	Импорт NumPy — основная библиотека для работы с массивами и математическими операциями.
2	<code>from config import SEED, AXIS_LIMIT</code>	Из файла <code>config.py</code> берём <code>SEED</code> (для воспроизводимости) и <code>AXIS_LIMIT</code> (граница координат).

2.2. Строки 4–11: `random_polynomial()`

Строка	Код	Пояснение
3	(пустая строка)	Разделитель между импортами и телом файла.
4	<code>def random_polynomial(seed=SEED):</code>	Объявляем функцию, генерирующую случайный полином 3-й степени. Параметр <code>seed</code> — для воспроизводимости.
5	<code> rng = np.random.default_rng(seed)</code>	Создаём генератор случайных чисел (новый NumPy API). Каждый вызов с тем же <code>seed</code> даёт те же числа.
6	<code> a, b, c, d = rng.uniform(-3, 3, 4)</code>	Генерируем 4 случайных коэффициента из равномерного распределения $[-3; 3]$ и распаковываем в <code>a, b, c, d</code> .
7	(пустая строка)	Разделитель внутри функции.
8	<code> def f(x):</code>	Объявляем вложенную функцию-замыкание (closure), которая «запоминает» коэффициенты <code>a, b, c, d</code> .
9	<code> return a * x**3 + b * x**2 + c * x + d</code>	Вычисляем полином $ax^3 + bx^2 + cx + d$. Работает поэлементно с массивами NumPy.
10	(пустая строка)	Разделитель.
11	<code> return f, (a, b, c, d)</code>	Возвращаем кортеж: саму функцию <code>f</code> и набор коэффициентов <code>(a, b, c, d)</code> для вывода в отчёт.

2.3. Строки 13–16: `trig_function()`

Строка	Код	Пояснение
12	(пустая строка)	Разделитель.
13	<code>def trig_function():</code>	Объявляем функцию-фабрику для тригонометрической функции.
14	<code> def f(x):</code>	Внутренняя функция — целевая зависимость.
15	<code> return x * np.sin(2 * np.pi * x)</code>	Формула $y = x \sin(2\pi x)$. <code>np.pi</code> ≈ 3.14159 . <code>np.sin</code> работает поэлементно.
16	<code> return f</code>	Возвращаем замыкание <code>f</code> .

2.4. Строки 18–27: generate_noise()

Строка	Код	Пояснение
17	(пустая строка)	Разделитель.
18	<code>def generate_noise(n, eps0, mode='uniform', sigma=None, seed=SEED):</code>	Генератор шума. n — число точек, eps0 — амплитуда, mode — тип распределения, sigma — σ для нормального.
19	<code>rng = np.random.default_rng(seed)</code>	Создаём локальный генератор с заданным seed .
20	<code>if sigma is None:</code>	Если σ не задана явно...
21	<code>sigma = eps0 / 3</code>	...по умолчанию берём $\sigma = \frac{\varepsilon_0}{3}$ — правило «трёх сигм»: 99.7 % значений попадут в $[-\varepsilon_0; \varepsilon_0]$.
22	<code>if mode == 'uniform':</code>	Проверяем: нужен ли равномерный шум?
23	<code>return rng.uniform(-eps0, eps0, n)</code>	Генерируем n значений из $\mathcal{U}(-\varepsilon_0, \varepsilon_0)$.
24	<code>if mode == 'normal':</code>	Проверяем: нужен ли нормальный шум?
25	<code>return rng.normal(0, sigma, n)</code>	Генерируем n значений из $\mathcal{N}(0, \sigma)$.
26	<code>raise ValueError(f'Unknown noise mode: {mode}')</code>	Если mode не „uniform“ и не „normal“ — бросаем ошибку с понятным сообщением.

2.5. Строки 28–33: generate_regression_sample()

Строка	Код	Пояснение
27	(пустая строка)	Разделитель.
28	<code>def generate_regression_sample(f, n, eps0, noise_mode='uniform', sigma=None, seed=SEED):</code>	Главная функция генерации регрессионной выборки. f — целевая функция, n — число точек, eps0 — амплитуда шума.
29	<code>rng = np.random.default_rng(seed)</code>	Генератор для x -координат.
30	<code>x = rng.uniform(-1, 1, n)</code>	Генерируем n случайных $x \in [-1, 1]$.
31	<code>noise = generate_noise(n, eps0, mode=noise_mode, sigma=sigma, seed=seed + 1)</code>	Создаём шум. $\text{seed} + 1$ — чтобы шум и x -координаты генерировались с разными seed , иначе были бы зависимы.
32	<code>y = f(x) + noise</code>	Вычисляем «зашумлённое» значение: $y = f(x) + \varepsilon$.
33	<code>return x.reshape(-1, 1), y.reshape(-1, 1)</code>	Возвращаем массивы формы $(n, 1)$. <code>reshape(-1, 1)</code> превращает вектор в столбец — нужно для PyTorch <code>Linear(1, ...)</code> .

2.6. Строки 35–50: make_circles()

Строка	Код	Пояснение
34	(пустая строка)	Разделитель.
35	<code>def make_circles(n=400, noise=0.08, factor=0.35, seed=SEED):</code>	Генерирует два концентрических кольца (Circles). factor — отношение радиусов.
36	<code>rng = np.random.default_rng(seed)</code>	Генератор.
37	<code>n_outer = n // 2</code>	Половина точек — внешнее кольцо.
38	<code>n_inner = n - n_outer</code>	Остальные — внутреннее. Если n нечётное, внутреннее получит на 1 больше.
39	(пустая строка)	Разделитель.

Строка	Код	Пояснение
40	<code>theta_outer = np.linspace(0, 2*np.pi, n_outer, endpoint=False)</code>	Равномерно располагаем углы от 0 до 2π для внешнего кольца. <code>endpoint=False</code> — точка 2π не включена (совпадает с 0).
41	<code>theta_inner = np.linspace(0, 2*np.pi, n_inner, endpoint=False)</code>	Аналогично для внутреннего кольца.
42	(пустая строка)	Разделитель.
43	<code>outer = np.column_stack([np.cos(theta_outer), np.sin(theta_outer)])</code>	Координаты точек внешнего кольца: $(\cos \theta, \sin \theta)$. <code>column_stack</code> — объединение двух столбцов в матрицу $(n_{\text{outer}} \times 2)$.
44	<code>inner = factor * np.column_stack([np.cos(...), np.sin(...)])</code>	Внутреннее кольцо — те же формулы, но умноженные на <code>factor</code> (масштабирование радиуса).
45	(пустая строка)	Разделитель.
46	<code>X = np.vstack([outer, inner])</code>	Склеиваем внешние и внутренние точки по вертикали $\rightarrow$ матрица $(n \times 2)$.
47	<code>y = np.concatenate([np.zeros(n_outer, dtype=int), np.ones(n_inner, dtype=int)])</code>	Метки: 0 для внешнего кольца, 1 для внутреннего.
48	<code>if noise > 0:</code>	Если шум задан...
49	<code>X += rng.normal(0, noise, X.shape)</code>	...добавляем гауссов шум к каждой координате. <code>X.shape</code> гарантирует, что размер шума совпадает с размером данных.
50	<code>return X, y</code>	Возвращаем матрицу признаков X и вектор меток y .

2.7. Строки 53–66: `make_xor()`

Строка	Код	Пояснение
51	(пустая строка)	Разделитель.
52	(пустая строка)	Дополнительный разделитель.
53	<code>def make_xor(n=400, noise=0.55, scale=2.2, seed=SEED):</code>	Генерирует XOR-подобный датасет — четыре облака в углах квадрата, противоположные пары одного класса.
54	<code>rng = np.random.default_rng(seed)</code>	Генератор.
55	<code>centers = np.array([[scale, scale], [scale, -scale], [-scale, scale], [-scale, -scale]])</code>	Четыре центра облаков. При <code>scale=2.2</code> : $(2.2, 2.2)$, $(2.2, -2.2)$, $(-2.2, 2.2)$, $(-2.2, -2.2)$.
56	<code>per = n // 4</code>	Базовое число точек на облако.
57	<code>rem = n % 4</code>	Остаток — распределяется по первым <code>rem</code> облакам.
58	<code>X_parts, y_parts = [], []</code>	Списки-аккумуляторы для частей X и y .
59	<code>for i, c in enumerate(centers):</code>	Перебираем центры с индексами.
60	<code>cnt = per + (1 if i < rem else 0)</code>	Число точек для данного облака: <code>per</code> , возможно $+1$ из остатка.
61	<code>X_parts.append(c + noise * rng.standard_normal((cnt, 2)))</code>	Генерируем <code>cnt</code> точек: центр + шум. <code>standard_normal</code> — $\mathcal{N}(0; 1)$, масштабирован <code>noise</code> .

Строка	Код	Пояснение
62	<code>y_parts.append(np.full(cnt, 0 if i in (0, 3) else 1, dtype=int))</code>	Метка: облака 0 и 3 (противоположные углы) → класс 0; облака 1 и 2 → класс 1. Это даёт XOR-структуру.
63	<code>X = np.vstack(X_parts)</code>	Склеиваем все части в одну матрицу.
64	<code>y = np.concatenate(y_parts)</code>	Склеиваем метки.
65	<code>perm = rng.permutation(len(X))</code>	Случайная перестановка индексов — перемешиваем данные.
66	<code>return X[perm], y[perm]</code>	Возвращаем перемешанные X и y.

2.8. Строки 69–86: `make_blobs()`

Строка	Код	Пояснение
67	(пустая строка)	Разделитель.
68	(пустая строка)	Дополнительный разделитель.
69	<code>def make_blobs(n=400, centers=None, cluster_std=0.9, seed=SEED):</code>	Генерирует два гауссовых облака. Самый простой для классификации датасет.
70	<code>rng = np.random.default_rng(seed)</code>	Генератор.
71	<code>if centers is None:</code>	Если центры не переданы...
72	<code>centers = np.array([[-2.5, -2.0], [2.3, 2.0]], dtype=float)</code>	...используем центры по умолчанию: $(-2.5, -2.0)$ и $(2.3, 2.0)$.
73	<code>else:</code>	Иначе...
74	<code>centers = np.asarray(centers, dtype=float)</code>	...приводим переданный список к NumPy-массиву.
75	<code>n_cls = len(centers)</code>	Число классов = число центров (у нас 2).
76	<code>base = n // n_cls</code>	Базовое число точек на класс.
77	<code>rem = n % n_cls</code>	Остаток.
78	<code>X_parts, y_parts = [], []</code>	Аккумуляторы.
79	<code>for k, c in enumerate(centers):</code>	Для каждого центра с индексом k...
80	<code>cnt = base + (1 if k < rem else 0)</code>	Число точек для данного облака.
81	<code>X_parts.append(c + cluster_std * rng.standard_normal((cnt, 2)))</code>	Точки = центр + гауссов шум с <code>std = cluster_std</code> .
82	<code>y_parts.append(np.full(cnt, k, dtype=int))</code>	Метка = индекс центра (0 или 1).
83	<code>X = np.vstack(X_parts)</code>	Склейка.
84	<code>y = np.concatenate(y_parts)</code>	Склейка меток.
85	<code>perm = rng.permutation(len(X))</code>	Перемешивание.
86	<code>return X[perm], y[perm]</code>	Возврат перемешанных данных.

2.9. Строки 88–103: `make_spiral()`

Строка	Код	Пояснение
87	(пустая строка)	Разделитель.
88	<code>def make_spiral(n=400, turns=1.5, radius=0.08, sweep=0.28, noise=0.08, seed=SEED):</code>	Генерирует двуспиральный датасет. <code>turns</code> — сколько оборотов, <code>sweep</code> — скорость расширения спирали, <code>radius</code> — начальный радиус.

Строка	Код	Пояснение
89	<code>rng = np.random.default_rng(seed)</code>	Генератор.
90	<code>n0 = n // 2</code>	Точек для спирали 0.
91	<code>n1 = n - n0</code>	Точек для спирали 1.
92	<code>theta0 = np.linspace(0, 2*np.pi*turns, n0)</code>	Углы от 0 до $2\pi \cdot \text{turns}$ для первой спирали. При $\text{turns}=1.5 \rightarrow$ от 0 до 3π .
93	<code>theta1 = np.linspace(0, 2*np.pi*turns, n1)</code>	То же для второй спирали.
94	<code>r0 = radius + sweep * theta0</code>	Радиус первой спирали растёт линейно с углом: $r = 0.08 + 0.28 \cdot \theta$. Это спираль Архимеда.
95	<code>r1 = radius + sweep * theta1</code>	Радиус второй спирали (те же параметры).
96	<code>X0 = np.column_stack([r0 * np.cos(theta0), r0 * np.sin(theta0)])</code>	Полярные $\rightarrow$ декартовы координаты: $x = r \cos \theta$, $y = r \sin \theta$.
97	<code>X1 = np.column_stack([r1*np.cos(theta1+np.pi), r1*np.sin(theta1+np.pi)])</code>	Вторая спираль сдвинута на π (180°): она закручена зеркально.
98	<code>X = np.vstack([X0, X1])</code>	Склейка спиралей.
99	<code>y = np.concatenate([np.zeros(n0, dtype=int), np.ones(n1, dtype=int)])</code>	Метки: первая спираль — 0, вторая — 1.
100	<code>if noise > 0:</code>	Если шум задан...
101	<code>X += rng.normal(0, noise, X.shape)</code>	...добавляем гауссов шум.
102	<code>perm = rng.permutation(len(X))</code>	Перемешиваем.
103	<code>return X[perm], y[perm]</code>	Возвращаем перемешанные данные.

2.10. Строки 105–110: CLS_DATASETS

Строка	Код	Пояснение
104	(пустая строка)	Разделитель.
105	<code>CLS_DATASETS = [</code>	Начало списка пар (название, функция-генератор). Используется в GUI.
106	<code>('Circles', make_circles),</code>	Пара: имя датасета и ссылка на функцию.
107	<code>('XOR', make_xor),</code>	Аналогично для XOR.
108	<code>('Blobs', make_blobs),</code>	Аналогично для Blobs.
109	<code>('Spiral', make_spiral),</code>	Аналогично для Spiral.
110	<code>]</code>	Конец списка.

2.11. Строки 112–116: train_test_split()

Строка	Код	Пояснение
111	(пустая строка)	Разделитель.
112	<code>def train_test_split(X, y, test_ratio=0.25, seed=SEED):</code>	Разделяет выборку на обучающую и тестовую. По умолчанию 75%/25%.
113	<code>rng = np.random.default_rng(seed)</code>	Генератор для перемешивания.
114	<code>idx = rng.permutation(len(X))</code>	Случайная перестановка индексов $[0, \dots, n-1]$.

Строка	Код	Пояснение
115	<code>cut = int(len(X) * (1 - test_ratio))</code>	Индекс разреза: первые <code>cut</code> — обучение, остальные — тест. При 280 объектах и 0.25: <code>cut = 210</code> .
116	<code>return X[idx[:cut]], X[idx[cut:]], y[idx[:cut]], y[idx[cut:]]</code>	Возвращаем <code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code> . Индексация через массив <code>idx</code> — элегантная перестановка без копий.

2.12. Строки 118–121: `standardize()`

Строка	Код	Пояснение
117	(пустая строка)	Разделитель.
118	<code>def standardize(X_train, X_test):</code>	Z-стандартизация (нормализация) признаков. Статистики считаются только по train .
119	<code>mean = X_train.mean(axis=0, keepdims=True)</code>	Среднее по каждому столбцу (признаку). <code>keepdims=True</code> сохраняет размерность (1, <i>d</i>) для broadcasting.
120	<code>std = X_train.std(axis=0, keepdims=True) + 1e-8</code>	Стандартное отклонение + 10^{-8} , чтобы не делить на 0.
121	<code>return (X_train - mean) / std, (X_test - mean) / std, mean, std</code>	Стандартизуем <code>train</code> и <code>test</code> одними и теми же <code>mean/std</code> . Возвращаем также <code>mean</code> и <code>std</code> для обратного преобразования.

2.13. Строки 123–126: `standardize_targets()`

Строка	Код	Пояснение
122	(пустая строка)	Разделитель.
123	<code>def standardize_targets(y_train, y_test):</code>	Аналогичная Z-стандартизация, но для целевых значений (<i>y</i>). Нужна только в задаче регрессии.
124	<code>mean = y_train.mean(axis=0, keepdims=True)</code>	Среднее таргетов по <code>train</code> .
125	<code>std = y_train.std(axis=0, keepdims=True) + 1e-8</code>	Std таргетов + защита от деления на 0.
126	<code>return (y_train - mean) / std, (y_test - mean) / std, mean, std</code>	Возвращаем стандартизованные таргеты и статистики для обратного преобразования при предсказании.

3. mlp.py — многослойный перцептрон

Файл содержит 120 строк. Реализует построение, обучение и метрики MLP на PyTorch.

3.1. Строки 1–9: Импорты и глобальные настройки

Строка	Код	Пояснение
1	<code>import copy</code>	Для глубокого копирования (<code>copy.deepcopy</code>) — нужен при сохранении лучшего состояния модели.
2	<code>import numpy as np</code>	NumPy — для работы с массивами при вычислении метрик.
3	<code>import torch</code>	Основной фреймворк глубокого обучения.
4	<code>import torch.nn as nn</code>	Модуль нейронных слоёв (<code>Linear</code> , <code>Sequential</code> , функции потерь, активации).
5	(пустая строка)	Разделитель.
6	<code>from config import SEED</code>	Импорт <code>seed</code> для фиксации случайности.
7	(пустая строка)	Разделитель.
8	<code>torch.manual_seed(SEED)</code>	Фиксируем <code>seed</code> PyTorch — все инициализации весов будут воспроизводимы.
9	<code>device = torch.device('cpu')</code>	Выбираем устройство — CPU. Можно заменить на <code>'cuda'</code> для GPU, но здесь это не нужно.

3.2. Строки 11–15: Словарь активаций

Строка	Код	Пояснение
10	(пустая строка)	Разделитель.
11	<code>ACTIVATIONS = {</code>	Словарь: строковое имя → класс активации PyTorch. Позволяет строить модель по имени.
12	<code>'sigmoid': nn.Sigmoid,</code>	$\sigma(x) = \frac{1}{1+e^{-x}}$. Гладкая, выходы в $(0, 1)$, склонна к «затуханию градиента».
13	<code>'tanh': nn.Tanh,</code>	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Выходы в $(-1, 1)$, центрирована — лучше <code>sigmoid</code> .
14	<code>'relu': nn.ReLU,</code>	$\text{ReLU}(x) = \max(0, x)$. Быстрая, не затухает для $x > 0$, но «мёртвые нейроны» при $x < 0$.
15	<code>}</code>	Конец словаря.

3.3. Строки 17–18: `to_tensor()`

Строка	Код	Пояснение
16	(пустая строка)	Разделитель.
17	<code>def to_tensor(array, dtype=torch.float32):</code>	Вспомогательная функция: NumPy-массив → PyTorch-тензор.
18	<code> return torch.tensor(np.asarray(array), dtype=dtype, device=device)</code>	<code>np.asarray</code> гарантирует NumPy-формат; <code>torch.tensor</code> создаёт тензор указанного типа на указанном устройстве.

3.4. Строки 20–31: build_mlp()

Строка	Код	Пояснение
19	(пустая строка)	Разделитель.
20	<code>def build_mlp(input_dim, output_dim, hidden_layers, hidden_dim, activation):</code>	Строит MLP. <code>input_dim/output_dim</code> — размерности входа/выхода; <code>hidden_layers</code> — число скрытых слоёв; <code>hidden_dim</code> — нейронов в каждом; <code>activation</code> — строка.
21	<code>layers = []</code>	Список, в который будем последовательно добавлять слои.
22	<code>in_f = input_dim</code>	Текущая входная размерность — начинаем с <code>input_dim</code> .
23	<code>act_cls = ACTIVATIONS[activation]</code>	Получаем класс активации из словаря по имени (например, <code>nn.ReLU</code>).
24	(пустая строка)	Разделитель.
25	<code>for _ in range(hidden_layers):</code>	Цикл по числу скрытых слоёв.
26	<code>layers.append(nn.Linear(in_f, hidden_dim))</code>	Добавляем линейный слой $y = Wx + b$ с <code>in_f</code> входами и <code>hidden_dim</code> выходами.
27	<code>layers.append(act_cls())</code>	Добавляем функцию активации (создаём экземпляр класса).
28	<code>in_f = hidden_dim</code>	Обновляем входную размерность для следующего слоя.
29	(пустая строка)	Разделитель.
30	<code>layers.append(nn.Linear(in_f, output_dim))</code>	Добавляем выходной линейный слой (без активации — «сырые» значения).
31	<code>return nn.Sequential(*layers).to(device)</code>	<code>nn.Sequential</code> объединяет все слои в единый модуль. <code>*layers</code> — распаковка списка. <code>.to(device)</code> — перенос на CPU.

3.5. Строки 33–63: train_model() — обучение с early stopping

Строка	Код	Пояснение
32	(пустая строка)	Разделитель.
33	<code>def train_model(model, X_train, y_train, X_val, y_val, task='regression', epochs=300, lr=0.01):</code>	Основная функция обучения. <code>task</code> определяет функцию потерь. Возвращает историю потерь.
34	<code>loss_fn = nn.MSELoss() if task == 'regression' else nn.CrossEntropyLoss()</code>	Регрессия $\rightarrow$ MSE ($\frac{1}{n} \sum (\hat{y}_i - y_i)^2$); классификация $\rightarrow$ кросс-энтропия ($-\sum y_k \log \hat{p}_k$).
35	<code>optimizer = torch.optim.Adam(model.parameters(), lr=lr)</code>	Оптимизатор Adam: адаптивный learning rate для каждого параметра. <code>model.parameters()</code> — все веса и сдвиги.
36	(пустая строка)	Разделитель.
37	<code>history = {'train_loss': [], 'val_loss': []}</code>	Словарь для записи потерь на каждой эпохе — для графиков.
38	<code>best_state = None</code>	Будем хранить лучшее состояние модели.
39	<code>best_val = float('inf')</code>	Инициализируем лучший val loss бесконечностью — любое реальное значение будет меньше.
40	(пустая строка)	Разделитель.

Строка	Код	Пояснение
41	<code>for _ in range(epochs):</code>	Цикл по эпохам.
42	<code>model.train()</code>	Переводим модель в режим обучения (влияет на Dropout/BatchNorm, у нас их нет, но это хорошая практика).
43	<code>optimizer.zero_grad()</code>	Обнуляем градиенты всех параметров перед новым шагом.
44	<code>out = model(X_train)</code>	Прямой проход (forward pass): вычисляем предсказания для обучающих данных.
45	<code>loss = loss_fn(out, y_train)</code>	Считаем функцию потерь.
46	<code>loss.backward()</code>	Обратное распространение ошибки (backpropagation) — вычисляем градиенты $\partial \frac{L}{\partial} w$ для каждого параметра.
47	<code>optimizer.step()</code>	Обновляем параметры: $w \leftarrow w - \text{lr} \cdot \nabla L$ (с адаптацией Adam).
48	(пустая строка)	Разделитель.
49	<code>model.eval()</code>	Переводим модель в режим оценки.
50	<code>with torch.no_grad():</code>	Отключаем вычисление градиентов — экономим память и время при валидации.
51	<code>val_out = model(X_val)</code>	Прямой проход на валидационных данных.
52	<code>val_loss = loss_fn(val_out, y_val)</code>	Считаем валидационную потерю.
53	(пустая строка)	Разделитель.
54	<code>history['train_loss'].append(float(loss.item()))</code>	Сохраняем train loss. <code>.item()</code> извлекает число из тензора-скаляра.
55	<code>history['val_loss'].append(float(val_loss.item()))</code>	Сохраняем val loss.
56	(пустая строка)	Разделитель.
57	<code>if val_loss.item() < best_val:</code>	Если текущая валидационная потеря лучше рекорда...
58	<code>best_val = float(val_loss.item())</code>	...обновляем рекорд.
59	<code>best_state = copy.deepcopy(model.state_dict())</code>	...сохраняем deep copy всех весов. <code>state_dict()</code> — словарь {имя_параметра: тензор}. Это early stopping .
60	(пустая строка)	Разделитель.
61	<code>if best_state is not None:</code>	Если в ходе обучения мы нашли хотя бы одно лучшее состояние...
62	<code>model.load_state_dict(best_state)</code>	...загружаем лучшие веса обратно в модель. Это и есть early stopping — возвращаемся к лучшей эпохе.
63	<code>return history</code>	Возвращаем историю для построения графиков.

3.6. Строки 65–86: `train_no_early_stop()`

Строка	Код	Пояснение
64	(пустая строка)	Разделитель.

Строка	Код	Пояснение
65	<code>def train_no_early_stop(model, X_train, y_train, X_val, y_val, epochs=2500, lr=0.01):</code>	Обучение без early stopping — для демонстрации переобучения. Модель обучается все <code>epochs</code> эпох без возврата.
66	<code>loss_fn = nn.MSELoss()</code>	Используем MSE (функция создана для регрессии в демо переобучения).
67	<code>optimizer = torch.optim.Adam(model.parameters(), lr=lr)</code>	Adam-оптимизатор.
68	<code>history = {'train_loss': [], 'val_loss': []}</code>	Словарь для истории.
69	(пустая строка)	Разделитель.
70	<code>for _ in range(epochs):</code>	Цикл по эпохам.
71	<code>model.train()</code>	Режим обучения.
72	<code>optimizer.zero_grad()</code>	Обнуляем градиенты.
73	<code>out = model(X_train)</code>	Прямой проход.
74	<code>loss = loss_fn(out, y_train)</code>	Расчёт потерь.
75	<code>loss.backward()</code>	Backpropagation.
76	<code>optimizer.step()</code>	Обновление весов.
77	(пустая строка)	Разделитель.
78	<code>model.eval()</code>	Режим оценки.
79	<code>with torch.no_grad():</code>	Без градиентов.
80	<code>val_out = model(X_val)</code>	Предсказания на val.
81	<code>val_loss = loss_fn(val_out, y_val)</code>	Val loss.
82	(пустая строка)	Разделитель.
83	<code>history['train_loss'].append(float(loss.item()))</code>	Сохраняем train loss.
84	<code>history['val_loss'].append(float(val_loss.item()))</code>	Сохраняем val loss.
85	(пустая строка)	Разделитель.
86	<code>return history</code>	Возвращаем историю. Без возврата к лучшим весам — модель в состоянии последней эпохи.

3.7. Строки 88–92: `predict_regression()`

Строка	Код	Пояснение
87	(пустая строка)	Разделитель.
88	<code>def predict_regression(model, X_t, y_mean, y_std):</code>	Предсказание для регрессии с обратной стандартизацией.
89	<code>model.eval()</code>	Режим оценки.
90	<code>with torch.no_grad():</code>	Без градиентов.
91	<code>pred = model(X_t).cpu().numpy()</code>	Прямой проход → NumPy. <code>.cpu()</code> — на всякий случай (у нас CPU). <code>.numpy()</code> — конвертация.
92	<code>return pred * y_std + y_mean</code>	Обратная стандартизация: $\hat{y}_{\text{orig}} = \hat{y}_{\text{std}} \cdot \sigma_y + \mu_y$. Восстанавливаем исходный масштаб.

3.8. Строки 94–98: predict_classes()

Строка	Код	Пояснение
93	(пустая строка)	Разделитель.
94	<code>def predict_classes(model, X_t):</code>	Предсказание классов (0 или 1).
95	<code>model.eval()</code>	Режим оценки.
96	<code>with torch.no_grad():</code>	Без градиентов.
97	<code>logits = model(X_t)</code>	Получаем «сырые» выходы (logits) — два числа на объект (для 2 классов).
98	<code>return torch.argmax(logits, dim=1).cpu().numpy()</code>	<code>argmax</code> по <code>dim=1</code> выбирает индекс максимального <code>logit</code> → метка класса. Результат — NumPy-массив.

3.9. Строки 100–101: mse_np()

Строка	Код	Пояснение
99	(пустая строка)	Разделитель.
100	<code>def mse_np(y_true, y_pred):</code>	Вычисляет MSE на NumPy — для итоговой метрики (не для обучения).
101	<code>return float(np.mean((np.asarray(y_true) - np.asarray(y_pred)) ** 2))</code>	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$. <code>np.asarray</code> гарантирует NumPy; <code>float(...)</code> — скаляр.

3.10. Строки 104–115: classification_metrics()

Строка	Код	Пояснение
102	(пустая строка)	Разделитель.
103	(пустая строка)	Дополнительный разделитель.
104	<code>def classification_metrics(y_true, y_pred):</code>	Вычисляет accuracy, precision, recall и confusion matrix.
105	<code>y_true = np.asarray(y_true).ravel()</code>	Приводим к одномерному NumPy-массиву. <code>.ravel()</code> — «разворачивает» в вектор.
106	<code>y_pred = np.asarray(y_pred).ravel()</code>	Аналогично для предсказаний.
107	<code>tn = int(np.sum((y_true == 0) & (y_pred == 0)))</code>	True Negatives: сколько объектов класса 0 правильно предсказаны как 0.
108	<code>tp = int(np.sum((y_true == 1) & (y_pred == 1)))</code>	True Positives: сколько объектов класса 1 правильно предсказаны как 1.
109	<code>fp = int(np.sum((y_true == 0) & (y_pred == 1)))</code>	False Positives: объекты класса 0, ошибочно предсказанные как 1.
110	<code>fn = int(np.sum((y_true == 1) & (y_pred == 0)))</code>	False Negatives: объекты класса 1, ошибочно предсказанные как 0.
111	<code>acc = (tp + tn) / max(tp + tn + fp + fn, 1)</code>	Accuracy = $\frac{TP + TN}{N}$. <code>max(..., 1)</code> — защита от деления на 0.
112	<code>prec = tp / max(tp + fp, 1)</code>	Precision = $\frac{TP}{TP + FP}$ — какую долю объявленных «класс 1» мы угадали.
113	<code>rec = tp / max(tp + fn, 1)</code>	Recall = $\frac{TP}{TP + FN}$ — какую долю реальных «класс 1» мы нашли.
114	<code>cm = np.array([[tn, fp], [fn, tp]])</code>	Confusion matrix 2 × 2: строки — истинные, столбцы — предсказанные.

Строка	Код	Пояснение
115	<pre> return {'accuracy': ..., 'precision': ..., 'recall': ..., 'confusion_matrix': cm}</pre>	Возвращаем словарь метрик. <code>float(...)</code> — для JSON-совместимости.

4. block1_main.py — регрессионные эксперименты

Файл содержит **229 строк**. Запускает перебор архитектур MLP для двух наборов данных (полином, тригонометрия), строит графики, демонстрирует переобучение.

4.1. Строки 1–9: Импорты

Строка	Код	Пояснение
1	<code>import copy</code>	Глубокое копирование — нужно для <code>copy.deepcopy(model)</code> при сохранении лучшей модели.
2	<code>import itertools</code>	Для <code>itertools.product</code> — декартово произведение (слои $\times$ нейроны).
3	<code>import os</code>	Для работы с путями (сохранение изображений).
4	<code>import numpy as np</code>	NumPy.
5	<code>import matplotlib.pyplot as plt</code>	Matplotlib — построение графиков.
6	(пустая строка)	Разделитель.
7	<code>from config import SEED, REG_N, REG_EPS0, REG_EPOCHS, REG_LR, DPI</code>	Импортируем нужные параметры из <code>config.py</code> .
8	<code>from generators import (random_polynomial, trig_function, ...)</code>	Из <code>generators.py</code> берём генераторы данных и утилиты: <code>generate_regression_sample</code> , <code>train_test_split</code> , <code>standardize</code> , <code>standardize_targets</code> .
9	<code>from mlp import (build_mlp, train_model, train_no_early_stop, predict_regression, mse_np, to_tensor)</code>	Из <code>mlp.py</code> берём всё, что связано с нейросетью.

4.2. Строки 11–28: Создание данных и константы

Строка	Код	Пояснение
10	(пустая строка)	Разделитель.
11	<code>poly_f, poly_coeffs = random_polynomial(seed=7)</code>	Создаём случайный полином с <code>seed=7</code> . <code>poly_f</code> — функция, <code>poly_coeffs</code> — кортеж (a, b, c, d) .
12	<code>trig_f = trig_function()</code>	Создаём тригонометрическую функцию $x \sin(2\pi x)$.
13	(пустая строка)	Разделитель.
14	<code>REGRESSION_DATASETS = [</code>	Начало списка словарей — каждый описывает один датасет.
15	<code>{</code>	Начало словаря для полиномиального датасета.
16	<code> 'name': 'Полином + равн. шум',</code>	Имя датасета для вывода в консоль и заголовки графиков.
17	<code> 'func': poly_f,</code>	Ссылка на функцию-полином (для сравнения с предсказанием на графике).
18	<code> 'data': generate_regression_sample(poly_f, REG_N, REG_EPS0, noise_mode='uniform', seed=11),</code>	Генерируем 280 точек с равномерным шумом $\varepsilon_0 = 0.18$. <code>seed=11</code> — фиксация. Возвращает (x, y) .
19	<code>},</code>	Конец словаря полиномиального датасета.

Строка	Код	Пояснение
20	{	Начало словаря для тригонометрического датасета.
21	'name': 'x·sin(2πx) + норм. шум',	Имя.
22	'func': trig_f,	Ссылка на $x \sin(2\pi x)$.
23	'data': generate_regression_sample(trig_f, REG_N, 0.12, noise_mode='normal', sigma=0.05, seed=13),	280 точек, нормальный шум $\sigma = 0.05$.
24	},	Конец словаря.
25	]	Конец списка REGRESSION_DATASETS.
26	(пустая строка)	Разделитель.
27	IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'images')	Путь к <code>images/</code> , рядом с текущим файлом. <code>__file__</code> — путь к <code>block1_main.py</code> , <code>abspath</code> — полный путь, <code>dirname</code> — директория.
28	DS_SLUG = {'Полином + равн. шум': 'poly', 'x·sin(2πx) + норм. шум': 'trig'}	Словарь коротких имён для файлов: <code>reg_poly_sigmoid.png</code> , <code>reg_trig_relu.png</code> и т.д.

4.3. Строки 30–74: run_regression_experiments()

Строка	Код	Пояснение
29	(пустая строка)	Разделитель.
30	def run_regression_experiments(datasets, activations=(...), epochs=REG_EPOCHS, lr=REG_LR):	Основная функция перебора. Проверяет все комбинации (датасет × активация × архитектура).
31	rows = []	Список-аккумулятор для всех результатов экспериментов.
32	best_models = {}	Словарь лучших моделей по ключу (dataset_name, activation).
33	archs = list(itertools.product([1,2,3], [1,2,3,4,5]))	Декартово произведение: 3 варианта скрытых слоёв × 5 вариантов нейронов = 15 архитектур.
34	(пустая строка)	Разделитель.
35	for ds in datasets:	Цикл по датасетам (полином, тригонометрия).
36	name = ds['name']	Извлекаем имя датасета.
37	x, y = ds['data']	Извлекаем данные: (x, y) — массивы формы (280, 1).
38	func = ds['func']	Целевая функция (для визуализации).
39	(пустая строка)	Разделитель.
40	X_tr, X_te, y_tr, y_te = train_test_split(x, y, seed=SEED)	Разбиение 75/25: 210 train / 70 test.
41	X_tr_s, X_te_s, x_m, x_s = standardize(X_tr, X_te)	Z-стандартизация X. x_m, x_s — среднее и std (для обратного преобразования).
42	y_tr_s, y_te_s, y_m, y_s = standardize_targets(y_tr, y_te)	Z-стандартизация Y. y_m, y_s — для обратного преобразования предсказаний.

Строка	Код	Пояснение
43	(пустая строка)	Разделитель.
44	<code>X_tr_t = to_tensor(X_tr_s)</code>	Преобразуем стандартизованные <code>X_train</code> в тензор PyTorch.
45	<code>X_te_t = to_tensor(X_te_s)</code>	<code>X_test</code> → тензор.
46	<code>y_tr_t = to_tensor(y_tr_s)</code>	<code>y_train</code> → тензор.
47	<code>y_te_t = to_tensor(y_te_s)</code>	<code>y_test</code> → тензор.
48	(пустая строка)	Разделитель.
49	<code>for act in activations:</code>	Цикл по активациям (sigmoid, tanh, relu).
50	<code>best = None</code>	Лучшая модель для текущей (датасет, активация).
51	<code>for h_layers, h_dim in archs:</code>	Перебор: <code>h_layers</code> ∈ {1,2,3}, <code>h_dim</code> ∈ {1,2,3,4,5}.
52	<code>model = build_mlp(1, 1, h_layers, h_dim, act)</code>	Строим MLP: 1 вход, 1 выход, <code>h_layers</code> скрытых, <code>h_dim</code> нейронов, активация <code>act</code> .
53	<code>hist = train_model(model, X_tr_t, y_tr_t, X_te_t, y_te_t, task='regression', ...)</code>	Обучаем модель с early stopping. Получаем историю потерь.
54	<code>pred = predict_regression(model, X_te_t, y_m, y_s)</code>	Предсказание на тесте с обратной стандартизацией.
55	<code>test_mse = mse_np(y_te, pred)</code>	Считаем MSE между оригинальными <code>y</code> и предсказанием.
56	(пустая строка)	Разделитель.
57	<code>rec = {</code>	Начало словаря-записи для текущего эксперимента.
58	<code>'dataset': name, 'activation': act,</code>	Имя датасета и активация.
59	<code>'linear_layers': h_layers + 1,</code>	Число линейных слоёв = скрытые + 1 (выходной). Если 2 скрытых → 3 линейных.
60	<code>'hidden_layers': h_layers, 'hidden_dim': h_dim,</code>	Число скрытых слоёв и нейронов.
61	<code>'test_mse': test_mse, 'history': hist,</code>	Метрика и история.
62	<code>'model': copy.deepcopy(model), 'func': func,</code>	Депер копия модели и ссылка на целевую функцию.
63	<code>'x_train': X_tr, 'x_test': X_te,</code>	Сохраняем данные для визуализации.
64	<code>'y_train': y_tr, 'y_test': y_te,</code>	Таргеты.
65	<code>'x_mean': x_m, 'x_std': x_s,</code>	Статистики X.
66	<code>'y_mean': y_m, 'y_std': y_s,</code>	Статистики Y.
67	<code>}</code>	Конец записи.
68	<code>rows.append(rec)</code>	Добавляем запись в общий список.

Строка	Код	Пояснение
69	<code>if best is None or test_mse < best['test_mse']:</code>	Если это первая модель или лучше предыдущей лучшей...
70	<code>best = rec</code>	...обновляем лучшую.
71	(пустая строка)	Разделитель.
72	<code>best_models[(name, act)] = best</code>	Сохраняем лучшую модель для комбинации (датасет, активация).
73	(пустая строка)	Разделитель.
74	<code>return rows, best_models</code>	Возвращаем все результаты и словарь лучших.

4.4. Строки 76–90: `print_pivot()`

Строка	Код	Пояснение
75	(пустая строка)	Разделитель.
76	<code>def print_pivot(rows, ds_name, act):</code>	Выводит сводную таблицу MSE в консоль для заданного датасета и активации.
77	<code>subset = [r for r in rows if r['dataset'] == ds_name and r['activation'] == act]</code>	Фильтруем записи: только нужный датасет + активация.
78	<code>all_layers = sorted({r['linear_layers'] for r in subset})</code>	Уникальные значения числа слоёв, отсортированные.
79	<code>all_dims = sorted({r['hidden_dim'] for r in subset})</code>	Уникальные числа нейронов.
80	(пустая строка)	Разделитель.
81	<code>col_w = 12</code>	Ширина столбца (12 символов).
82	<code>header = f"{'Слоёв':<8}" + ''.join(f"{'N='+str(d):<{col_w}}" for d in all_dims)</code>	Формируем заголовок: «Слоёв» + «N=1», «N=2» и т.д.
83	<code>print(header)</code>	Печатаем заголовок.
84	<code>print('-' * len(header))</code>	Горизонтальная линия.
85	<code>for nl in all_layers:</code>	Для каждого числа слоёв...
86	<code>line = f'{nl:<8}'</code>	...начинаем строку с числа слоёв.
87	<code>for hd in all_dims:</code>	Для каждого числа нейронов...
88	<code>val = next((r['test_mse'] for r in subset if ...), None)</code>	...ищем MSE. <code>next(..., None)</code> — если не найдено, <code>None</code> .
89	<code>line += f'{val:<{col_w}.6f}' if val is not None else f'{"-":<{col_w}}'</code>	Форматируем: 6 знаков после точки или прочерк.
90	<code>print(line)</code>	Печатаем строку таблицы.

4.5. Строки 92–127: `plot_best_regression()`

Строка	Код	Пояснение
91	(пустая строка)	Разделитель.
92	<code>def plot_best_regression(record, save_path=None):</code>	Строит два графика для лучшей модели: loss-кривые и аппроксимацию.
93	<code>model = record['model']</code>	Извлекаем обученную модель.
94	<code>func = record['func']</code>	Целевая функция.

Строка	Код	Пояснение
95	<code>x_m, x_s = record['x_mean'], record['x_std']</code>	Статистики X для стандартизации.
96	<code>y_m, y_s = record['y_mean'], record['y_std']</code>	Статистики Y для обратной стандартизации.
97	(пустая строка)	Разделитель.
98	<code>x_dense = np.linspace(-1, 1, 400).reshape(-1, 1)</code>	400 равномерных точек по x для плавной кривой.
99	<code>x_dense_s = (x_dense - x_m) / x_s</code>	Стандартизуем плотную сетку теми же статистиками.
100	<code>y_dense = predict_regression(model, to_tensor(x_dense_s), y_m, y_s)</code>	Предсказание MLP на плотной сетке с обратной стандартизацией.
101	(пустая строка)	Разделитель.
102	<code>fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))</code>	Фигура с двумя субплотами: 14×4.5 дюймов.
103	(пустая строка)	Разделитель.
104	<code>axes[0].plot(record['history'] ['train_loss'], label='train loss')</code>	Левый график: кривая train loss.
105	<code>axes[0].plot(record['history'] ['val_loss'], label='test loss')</code>	Кривая val/test loss.
106	<code>axes[0].set_title(f"Loss {record['activation']} ...")</code>	Заголовок с параметрами модели.
107	<code>axes[0].set_xlabel('epoch')</code>	Подпись оси X.
108	<code>axes[0].set_ylabel('loss')</code>	Подпись оси Y.
109	<code>axes[0].legend()</code>	Легенда.
110	<code>axes[0].grid(True, alpha=0.3)</code>	Полупрозрачная сетка.
111	(пустая строка)	Разделитель.
112	<code>axes[1].scatter(record['x_train'] [:,0], record['y_train'][:,0], s=16, alpha=0.65, label='train')</code>	Правый график: точки обучающей выборки.[:,0] — берём первый (единственный) столбец.
113	<code>axes[1].scatter(record['x_test'][:,0], record['y_test'][:,0], s=20, alpha=0.8, label='test')</code>	Точки тестовой выборки.
114	<code>axes[1].plot(x_dense[:,0], func(x_dense[:,0]), color='black', lw=2, label='истинная f(x)')</code>	Чёрная кривая — истинная функция.
115	<code>axes[1].plot(x_dense[:,0], y_dense[:,0], color='crimson', lw=2, label='MLP')</code>	Красная кривая — предсказание MLP.
116	<code>axes[1].set_title(f"... test MSE = {record['test_mse']:.4f}")</code>	Заголовок с MSE.
117	<code>axes[1].set_xlabel('x')</code>	Подпись X.
118	<code>axes[1].set_ylabel('y')</code>	Подпись Y.
119	<code>axes[1].legend()</code>	Легенда.
120	<code>axes[1].grid(True, alpha=0.3)</code>	Сетка.
121	(пустая строка)	Разделитель.
122	<code>plt.tight_layout()</code>	Автоматическая подгонка отступов.
123	<code>if save_path:</code>	Если задан путь для сохранения...

Строка	Код	Пояснение
124	<code>os.makedirs(os.path.dirname(save_path), exist_ok=True)</code>	...создаём директорию (если не существует). <code>exist_ok=True</code> — не ошибка, если уже есть.
125	<code>fig.savefig(save_path, dpi=DPI, bbox_inches='tight')</code>	...сохраняем PNG. <code>bbox_inches='tight'</code> — обрезаем лишние поля.
126	<code>plt.show()</code>	Показываем окно.
127	<code>plt.close()</code>	Закрываем фигуру — освобождаем память.

4.6. Строки 129–194: `run_overfit_demo()`

Строка	Код	Пояснение
128	(пустая строка)	Разделитель.
129	<code>def run_overfit_demo():</code>	Демонстрация переобучения: маленькая выборка + сложная модель + много эпох.
130	<code>ds = next(d for d in REGRESSION_DATASETS if 'sin' in d['name'])</code>	Берём тригонометрический датасет (где есть „sin“ в названии).
131	<code>x, y = ds['data']</code>	Данные.
132	<code>func = ds['func']</code>	Функция.
133	(пустая строка)	Разделитель.
134	<code>X_tr_full, X_te, y_tr_full, y_te = train_test_split(x, y, seed=SEED)</code>	Полное разбиение 75/25.
135	(пустая строка)	Разделитель.
136	<code>rng = np.random.default_rng(SEED)</code>	Генератор для выбора подмножества.
137	<code>small_idx = rng.choice(len(X_tr_full), size=10, replace=False)</code>	Выбираем 10 случайных объектов из train. <code>replace=False</code> — без повторений.
138	<code>X_tr = X_tr_full[small_idx]</code>	Маленький train X (10 точек).
139	<code>y_tr = y_tr_full[small_idx]</code>	Маленький train y.
140	(пустая строка)	Разделитель.
141	<code>X_tr_s, X_te_s, x_m, x_s = standardize(X_tr, X_te)</code>	Стандартизация (по 10 точкам — очень нестабильные статистики).
142	<code>y_tr_s, y_te_s, y_m, y_s = standardize_targets(y_tr, y_te)</code>	Стандартизация Y.
143	(пустая строка)	Разделитель.
144	<code>X_tr_t = to_tensor(X_tr_s)</code>	Тензор X_train.
145	<code>X_te_t = to_tensor(X_te_s)</code>	Тензор X_test.
146	<code>y_tr_t = to_tensor(y_tr_s)</code>	Тензор y_train.
147	<code>y_te_t = to_tensor(y_te_s)</code>	Тензор y_test.
148	(пустая строка)	Разделитель.
149	<code>model = build_mlp(1, 1, hidden_layers=8, hidden_dim=5, activation='tanh')</code>	Строим избыточно сложную модель: 8 скрытых слоёв × 5 нейронов = 9 линейных слоёв. Для 10 точек — явный перебор.
150	<code>hist = train_no_early_stop(model, X_tr_t, y_tr_t, X_te_t, y_te_t, epochs=5000, lr=0.01)</code>	Обучаем 5000 эпох без early stopping — нарочно даём переобучиться.
151	(пустая строка)	Разделитель.

Строка	Код	Пояснение
152	<code>pred_tr = predict_regression(model, X_tr_t, y_m, y_s)</code>	Предсказания на train.
153	<code>pred_te = predict_regression(model, X_te_t, y_m, y_s)</code>	Предсказания на test.
154	<code>tr_mse = mse_np(y_tr, pred_tr)</code>	Train MSE — ожидается очень низкий (модель «запомнила» 10 точек).
155	<code>te_mse = mse_np(y_te, pred_te)</code>	Test MSE — ожидается высокий (Test >> Train) — признак переобучения.
156	(пустая строка)	Разделитель.
157	<code>best_epoch = int(np.argmin(hist['val_loss'])) + 1</code>	<code>argmin</code> находит индекс минимального val loss. +1 — нумерация эпох с 1.
158	(пустая строка)	Разделитель.
159	<code>print(f'\nПереобучение на $x \cdot \sin(2\pi x)$')</code>	Заголовок вывода.
160	<code>print(f' Train объектов: {len(X_tr)}')</code>	Выводим: 10.
161	<code>print(f' Train MSE: {tr_mse:.6f}')</code>	Train MSE с 6 знаками.
162	<code>print(f' Test MSE: {te_mse:.6f}')</code>	Test MSE с 6 знаками.
163	<code>print(f' Лучшая эпоха по val loss: {best_epoch}')</code>	Эпоха, когда val loss был минимален.
164	(пустая строка)	Разделитель.
165	<code>x_dense = np.linspace(-1, 1, 500).reshape(-1, 1)</code>	500 точек для плавной кривой.
166	<code>x_dense_s = (x_dense - x_m) / x_s</code>	Стандартизация.
167	<code>y_dense = predict_regression(model, to_tensor(x_dense_s), y_m, y_s)</code>	Предсказание MLP на плотной сетке.
168	(пустая строка)	Разделитель.
169	<code>fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))</code>	Два графика.
170	(пустая строка)	Разделитель.
171	<code>axes[0].plot(hist['train_loss'], label='train loss')</code>	Кривая train loss.
172	<code>axes[0].plot(hist['val_loss'], label='test loss')</code>	Кривая test loss — расходится после минимума.
173	<code>axes[0].axvline(best_epoch - 1, color='crimson', ls='--', label='минимум test loss')</code>	Вертикальная красная пунктирная линия в точке минимума val loss.
174	<code>axes[0].set_title('Переобучение: train / test loss')</code>	Заголовок.
175	<code>axes[0].set_xlabel('epoch')</code>	Подпись X.
176	<code>axes[0].set_ylabel('loss')</code>	Подпись Y.
177	<code>axes[0].legend()</code>	Легенда.
178	<code>axes[0].grid(True, alpha=0.3)</code>	Сетка.
179	(пустая строка)	Разделитель.
180	<code>axes[1].scatter(X_tr[:,0], y_tr[:,0], s=28, alpha=0.85, label='train')</code>	10 обучающих точек (крупные).
181	<code>axes[1].scatter(X_te[:,0], y_te[:,0], s=24, alpha=0.65, label='test')</code>	Тестовые точки.

Строка	Код	Пояснение
182	<code>axes[1].plot(x_dense[:,0], func(x_dense[:,0]), color='black', lw=2, label='истинная f(x)')</code>	Чёрная кривая — истинная функция.
183	<code>axes[1].plot(x_dense[:,0], y_dense[:,0], color='crimson', lw=2, label='MLP (переобучение)')</code>	Красная кривая — модель, которая «изгибается» через 10 точек вместо правильной формы.
184	<code>axes[1].set_title('Переобученная аппроксимация $x \cdot \sin(2\pi x)$')</code>	Заголовок.
185	<code>axes[1].set_xlabel('x')</code>	Подпись X.
186	<code>axes[1].set_ylabel('y')</code>	Подпись Y.
187	<code>axes[1].legend()</code>	Легенда.
188	<code>axes[1].grid(True, alpha=0.3)</code>	Сетка.
189	(пустая строка)	Разделитель.
190	<code>plt.tight_layout()</code>	Подгонка.
191	<code>os.makedirs(IMG_DIR, exist_ok=True)</code>	Создаём папку images.
192	<code>fig.savefig(os.path.join(IMG_DIR, 'overfit.png'), dpi=DPI, bbox_inches='tight')</code>	Сохраняем как overfit.png.
193	<code>plt.show()</code>	Показываем.
194	<code>plt.close()</code>	Закрываем.

4.7. Строки 196–229: main()

Строка	Код	Пояснение
195	(пустая строка)	Разделитель.
196	<code>def main():</code>	Точка входа.
197	<code>print(f'Коэффициенты полинома: {poly_coeffs}')</code>	Выводим коэффициенты (a, b, c, d) — для записи в отчёт.
198	<code>print(f'Запуск экспериментов по регрессии (epochs={REG_EPOCHS}, lr={REG_LR})...')</code>	Информационное сообщение.
199	(пустая строка)	Разделитель.
200	<code>rows, best = run_regression_experiments(REGRESSION_DATASETS)</code>	Запуск экспериментов (2 датасета $\times$ 3 активации $\times$ 15 архитектур).
201	(пустая строка)	Разделитель.
202	<code>activations = ['sigmoid', 'tanh', 'relu']</code>	Список активаций для вывода.
203	<code>ds_names = [d['name'] for d in REGRESSION_DATASETS]</code>	Список имён датасетов.
204	(пустая строка)	Разделитель.
205	<code>for ds_name in ds_names:</code>	Для каждого датасета...
206	<code>print('\n' + '=' * 70)</code>	Разделительная линия из 70 символов =.
207	<code>print(ds_name)</code>	Имя датасета.
208	<code>print('=' * 70)</code>	Ещё линия.
209	<code>for act in activations:</code>	Для каждой активации...
210	<code>print(f'\n Активация: {act} (Test MSE)')</code>	Подзаголовок.

Строка	Код	Пояснение
211	<code>print(' ' + '-' * 62)</code>	Линия.
212	<code>print_pivot(rows, ds_name, act)</code>	Печатаем сводную таблицу MSE.
213	(пустая строка)	Разделитель.
214	<code>print('\n\n— Лучшие модели по регрессии —')</code>	Заголовок раздела.
215	<code>for (ds_name, act), rec in best.items():</code>	Перебираем лучшие модели по ключу (д-тасет, активация).
216	<code>print(f' {ds_name} {act}: слоёв={rec["linear_layers"]}, нейронов={rec["hidden_dim"]}, MSE={rec["test_mse"]:.6f}')</code>	Выводим характеристики лучшей модели.
217–218	<code>(slug = ..., plot_best_regression(rec, ...))</code>	Получаем короткое имя и строим/сохраняем графики лучшей модели.
219	(пустая строка)	Разделитель.
220	<code>slug = DS_SLUG.get(ds_name, ds_name)</code>	Получаем 'poly' или 'trig' для имени файла.
221	<code>plot_best_regression(rec, save_path=os.path.join(IMG_DIR, f'reg_{slug}_{act}.png'))</code>	Строим и сохраняем: <code>reg_poly_sigmoid.png</code> , <code>reg_trig_relu.png</code> и т.д.
222	(пустая строка)	Разделитель.
223	<code>print('\n\n— Демонстрация переобучения —')</code>	Заголовок.
224	<code>run_overfit_demo()</code>	Запуск демо переобучения.
225	(пустая строка)	Разделитель.
226	(пустая строка)	Дополнительный разделитель.
227	<code>if __name__ == '__main__':</code>	Стандартная проверка: файл запущен как скрипт (а не импортирован)?
228	<code>main()</code>	Вызываем <code>main()</code> .

5. block2_main.py — классификационные эксперименты

Файл содержит **310 строк**. Запускает классификацию, визуализирует границы решений, ищет минимальный размер выборки и проводит кросс-валидацию.

5.1. Строки 1–10: Импорты

Строка	Код	Пояснение
1	<code>import copy</code>	Глубокое копирование моделей.
2	<code>import itertools</code>	Декартово произведение архитектур.
3	<code>import os</code>	Работа с путями.
4	<code>import numpy as np</code>	NumPy.
5	<code>import matplotlib.pyplot as plt</code>	Matplotlib.
6	<code>import torch</code>	PyTorch — нужен для <code>torch.long</code> при создании меток.
7	(пустая строка)	Разделитель.
8	<code>from config import (SEED, CLS_N, CLS_NOISE, CLS_EPOCHS, CLS_LR, CV_K, DPI)</code>	Классификационные параметры из конфига.
9	<code>from generators import (CLS_DATASETS, make_circles, make_xor, make_blobs, make_spiral, train_test_split, standardize)</code>	Генераторы и утилиты.
10	<code>from mlp import (build_mlp, train_model, predict_classes, classification_metrics, to_tensor)</code>	Функции MLP для классификации.

5.2. Строки 12–24: `load_cls_datasets()`

Строка	Код	Пояснение
11	(пустая строка)	Разделитель.
12	<code>IMG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'images')</code>	Путь к файлам изображений.
13	(пустая строка)	Разделитель.
14	<code>def load_cls_datasets(n=CLS_N, noise=CLS_NOISE):</code>	Создаёт список из 4 классификационных датасетов.
15	<code> return [</code>	Начало списка.
16	<code> {'name': 'Circles',</code>	Словарь для Circles.
17	<code> 'data': make_circles(n=n, noise=noise, seed=21)},</code>	500 точек двух концентрических колец с <code>seed=21</code> .
18	<code> {'name': 'XOR',</code>	Словарь для XOR.
19	<code> 'data': make_xor(n=n, noise=0.55, seed=22)},</code>	XOR с <code>noise=0.55</code> (4 облака).
20	<code> {'name': 'Blobs',</code>	Словарь для Blobs.
21	<code> 'data': make_blobs(n=n, seed=23)},</code>	Два гауссовых облака.
22	<code> {'name': 'Spiral',</code>	Словарь для Spiral.
23	<code> 'data': make_spiral(n=n, noise=noise, seed=24)},</code>	Двуспиральный датасет — самый сложный.
24	<code>]</code>	Конец списка.

5.3. Строки 26–68: run_classification_experiments()

Строка	Код	Пояснение
25	(пустая строка)	Разделитель.
26	<code>def run_classification_experiments(datasets, activations=(...), epochs=CLS_EPOCHS, lr=CLS_LR):</code>	Основной перебор архитектур для классификации.
27	<code>rows = []</code>	Аккумулятор всех результатов.
28	<code>best_models = {}</code>	Лучшие модели по ключу (имя, активация).
29	<code>archs = list(itertools.product([1,2,3], [1,2,3,4,5]))</code>	15 архитектур.
30	(пустая строка)	Разделитель.
31	<code>for ds in datasets:</code>	Цикл по 4 датасетам.
32	<code>name = ds['name']</code>	Имя.
33	<code>X, y = ds['data']</code>	Данные: X — матрица (500×2), y — вектор из 0 и 1.
34	<code>X_tr, X_te, y_tr, y_te = train_test_split(X, y, seed=SEED)</code>	Разбиение 75/25.
35	<code>X_tr_s, X_te_s, x_m, x_s = standardize(X_tr, X_te)</code>	Z-стандартизация признаков . Метки не стандартизуем!
36	(пустая строка)	Разделитель.
37	<code>X_tr_t = to_tensor(X_tr_s)</code>	Тензор X_{train} .
38	<code>X_te_t = to_tensor(X_te_s)</code>	Тензор X_{test} .
39	<code>y_tr_t = to_tensor(y_tr, dtype=torch.long)</code>	Метки $\rightarrow$ long (int64). CrossEntropyLoss требует именно long.
40	<code>y_te_t = to_tensor(y_te, dtype=torch.long)</code>	Аналогично для теста.
41	(пустая строка)	Разделитель.
42	<code>for act in activations:</code>	Цикл по 3 активациям.
43	<code>best = None</code>	Лучшая модель для текущей комбинации.
44	<code>for h_layers, h_dim in archs:</code>	Перебор 15 архитектур.
45	<code>model = build_mlp(2, 2, h_layers, h_dim, act)</code>	MLP: 2 входа (x_1, x_2), 2 выхода (logits для 2 классов).
46	<code>hist = train_model(model, X_tr_t, y_tr_t, X_te_t, y_te_t, task='classification', ...)</code>	Обучение с CrossEntropyLoss и early stopping.
47	<code>pred = predict_classes(model, X_te_t)</code>	Предсказание классов на тесте.
48	<code>metrics = classification_metrics(y_te, pred)</code>	Accuracy, precision, recall, confusion matrix.
49	(пустая строка)	Разделитель.
50	<code>rec = {</code>	Начало записи.
51	<code>'dataset': name, 'activation': act,</code>	Имя и активация.
52	<code>'linear_layers': h_layers + 1,</code>	Число линейных слоёв.

Строка	Код	Пояснение
53	<code>'hidden_layers': h_layers, 'hidden_dim': h_dim,</code>	Параметры архитектуры.
54	<code>'test_accuracy': metrics['accuracy'],</code>	Ассурасу — основная метрика.
55	<code>'history': hist,</code>	История потерь.
56	<code>'model': copy.deepcopy(model),</code>	Копия модели.
57	<code>'X_train': X_tr, 'X_test': X_te,</code>	Данные.
58	<code>'y_train': y_tr, 'y_test': y_te,</code>	Метки.
59	<code>'x_mean': x_m, 'x_std': x_s,</code>	Статистики стандартизации.
60	<code>'confusion_matrix': metrics['confusion_matrix'],</code>	Confusion matrix.
61	<code>}</code>	Конец записи.
62	<code>rows.append(rec)</code>	Добавление.
63	<code>if best is None or metrics['accuracy'] > best['test_accuracy']:</code>	Ассурасу больше → лучше (в отличие от MSE).
64	<code>best = rec</code>	Обновляем лучшую.
65	<code>(пустая строка)</code>	Разделитель.
66	<code>best_models[(name, act)] = best</code>	Сохраняем.
67	<code>(пустая строка)</code>	Разделитель.
68	<code>return rows, best_models</code>	Возвращаем.

5.4. Строки 70–84: `print_cls_pivot()`

Строка	Код	Пояснение
69	<code>(пустая строка)</code>	Разделитель.
70	<code>def print_cls_pivot(rows, ds_name, act):</code>	Аналогична <code>print_pivot()</code> из <code>block1</code> , но выводит accuracy (4 знака) вместо MSE (6 знаков).
71	<code>subset = [r for r in rows if r['dataset'] == ds_name and r['activation'] == act]</code>	Фильтрация.
72	<code>all_layers = sorted({r['linear_layers'] for r in subset})</code>	Уникальные слои.
73	<code>all_dims = sorted({r['hidden_dim'] for r in subset})</code>	Уникальные нейроны.
74	<code>(пустая строка)</code>	Разделитель.
75	<code>col_w = 12</code>	Ширина столбца.
76	<code>header = f"{'Слоёв':<8}" + ''.join(f"{'N='+str(d):<{col_w}}" for d in all_dims)</code>	Заголовок.
77	<code>print(header)</code>	Печать.
78	<code>print('-' * len(header))</code>	Линия.
79	<code>for nl in all_layers:</code>	Цикл по строкам.

Строка	Код	Пояснение
80	<code>line = f'{nl:<8}'</code>	Начало строки.
81	<code>for hd in all_dims:</code>	Цикл по столбцам.
82	<code>val = next((r['test_accuracy'] for r in subset if ...), None)</code>	Ищем ассурасу.
83	<code>line += f'{val:<{col_w}.4f}' if val is not None else f'{"-":<{col_w}}'</code>	4 знака после точки.
84	<code>print(line)</code>	Печать строки.

5.5. Строки 86–133: `plot_decision_boundary()`

Строка	Код	Пояснение
85	(пустая строка)	Разделитель.
86	<code>def plot_decision_boundary(record, save_path=None):</code>	Строит 3 графика: loss, граница решений, confusion matrix.
87	<code>model = record['model']</code>	Модель.
88	<code>X_tr, X_te = record['X_train'], record['X_test']</code>	Данные.
89	<code>y_tr, y_te = record['y_train'], record['y_test']</code>	Метки.
90	<code>x_m, x_s = record['x_mean'], record['x_std']</code>	Статистики.
91	(пустая строка)	Разделитель.
92	<code>X_all = np.vstack([X_tr, X_te])</code>	Объединяем train + test для определения диапазона осей.
93	<code>lo = X_all.min(axis=0) - 0.5</code>	Нижняя граница по каждой оси (с отступом 0.5).
94	<code>hi = X_all.max(axis=0) + 0.5</code>	Верхняя граница.
95	(пустая строка)	Разделитель.
96	<code>xx, yy = np.meshgrid(np.linspace(lo[0], hi[0], 250), np.linspace(lo[1], hi[1], 250))</code>	Сетка 250×250 для заливки решающей границы. <code>meshgrid</code> создаёт координатные матрицы.
97	<code>grid = np.column_stack([xx.ravel(), yy.ravel()])</code>	«Разворачиваем» сетку в матрицу (62500 × 2).
98	<code>grid_s = (grid - x_m) / x_s</code>	Стандартизуем сетку теми же mean/std, что и train.
99	<code>zz = predict_classes(model, to_tensor(grid_s)).reshape(xx.shape)</code>	Предсказываем классы → возвращаем в форму 250×250 для <code>contourf</code> .
100	(пустая строка)	Разделитель.
101	<code>fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))</code>	Три субплота.
102	(пустая строка)	Разделитель.
103	<code>axes[0].plot(record['history'] ['train_loss'], label='train loss')</code>	Кривая train loss.
104	<code>axes[0].plot(record['history'] ['val_loss'], label='test loss')</code>	Кривая test loss.
105	<code>axes[0].set_title(f"Loss {record['activation']} слоёв=... нейронов=...")</code>	Заголовок.

Строка	Код	Пояснение
106	<code>axes[0].set_xlabel('epoch')</code>	Подпись X.
107	<code>axes[0].set_ylabel('loss')</code>	Подпись Y.
108	<code>axes[0].legend()</code>	Легенда.
109	<code>axes[0].grid(True, alpha=0.3)</code>	Сетка.
110	(пустая строка)	Разделитель.
111	<code>axes[1].contourf(xx, yy, zz, levels=2, alpha=0.35, cmap='coolwarm')</code>	Заливка решающей границы. <code>contourf</code> — заполненный контурный график. <code>levels=2</code> — два класса.
112	<code>axes[1].scatter(X_te[y_te==0, 0], X_te[y_te==0, 1], s=18, label='Класс 0')</code>	Точки класса 0 на тесте.
113	<code>axes[1].scatter(X_te[y_te==1, 0], X_te[y_te==1, 1], s=18, label='Класс 1')</code>	Точки класса 1.
114	<code>axes[1].set_title(f"... accuracy = {record['test_accuracy']:.3f}")</code>	Заголовок с accuracy.
115	<code>axes[1].legend()</code>	Легенда.
116	<code>axes[1].grid(True, alpha=0.3)</code>	Сетка.
117	(пустая строка)	Разделитель.
118	<code>cm = record['confusion_matrix']</code>	Извлекаем confusion matrix 2×2 .
119	<code>axes[2].imshow(cm, cmap='Blues')</code>	Отображаем как цветную матрицу (тепловая карта).
120	<code>for i in range(2):</code>	Для каждой строки (0, 1)...
121	<code>for j in range(2):</code>	...для каждого столбца...
122	<code>axes[2].text(j, i, str(cm[i, j]), ha='center', va='center', color='black', fontsize=13)</code>	...пишем число (TN/FP/FN/TP) в центре ячейки.
123	<code>axes[2].set_xticks([0, 1])</code>	Метки оси X: 0 и 1.
124	<code>axes[2].set_yticks([0, 1])</code>	Метки оси Y.
125	<code>axes[2].set_xlabel('predicted')</code>	Подпись X.
126	<code>axes[2].set_ylabel('true')</code>	Подпись Y.
127	<code>axes[2].set_title('Confusion matrix')</code>	Заголовок.
128	(пустая строка)	Разделитель.
129	<code>plt.tight_layout()</code>	Подгонка отступов.
130	<code>if save_path:</code>	Если задан путь...
131	<code>os.makedirs(os.path.dirname(save_path), exist_ok=True)</code>	...создаём директорию.
132	<code>fig.savefig(save_path, dpi=DPI, bbox_inches='tight')</code>	...сохраняем PNG.
133	<code>plt.show()</code>	Показываем.
134	<code>plt.close()</code>	Закрываем.

5.6. Строки 136–166: `estimate_min_train_size()`

Строка	Код	Пояснение
135	(пустая строка)	Разделитель.

Строка	Код	Пояснение
136	<code>def estimate_min_train_size(datasets, activation='tanh', linear_layers=4, hidden_dim=5, target_acc=0.90, ...):</code>	Ищет минимальную долю train, при которой ассигасу $\geq 90\%$. Фиксированная архитектура: tanh, 4 линейных слоя, 5 нейронов.
137	<code>fracs = np.arange(0.10, 0.96, 0.05)</code>	Доли train: 0.10, 0.15, 0.20, ..., 0.95 (18 значений).
138	<code>summary = []</code>	Аккумулятор.
139	(пустая строка)	Разделитель.
140	<code>for ds in datasets:</code>	Для каждого из 4 датасетов...
141	<code>name = ds['name']</code>	Имя.
142	<code>X, y = ds['data']</code>	Данные.
143	<code>X = np.asarray(X, dtype=np.float32)</code>	Приведение типа (float32 для PyTorch).
144	<code>y = np.asarray(y, dtype=np.int64)</code>	int64 для CrossEntropyLoss.
145	(пустая строка)	Разделитель.
146	<code>best_frac = None</code>	Минимальная доля, пока не найдена.
147	<code>for frac in fracs:</code>	Перебор от 10 % до 95 %.
148	<code>test_ratio = 1.0 - float(frac)</code>	$\text{test_ratio} = 1 - \text{frac}$. Например: $\text{frac}=0.10 \rightarrow 90\%$ тест.
149	<code>X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_ratio=test_ratio, seed=SEED)</code>	Разбиение с заданной долей.
150	<code>X_tr_s, X_te_s, _, _ = standardize(X_tr, X_te)</code>	Стандартизация. <code>_</code> , <code>_</code> — mean и std не нужны дальше.
151	(пустая строка)	Разделитель.
152	<code>model = build_mlp(2, 2, linear_layers-1, hidden_dim, activation)</code>	Строим модель. <code>linear_layers-1 = 3</code> скрытых слоя.
153	<code>train_model(model, to_tensor(X_tr_s), to_tensor(y_tr, torch.long), to_tensor(X_te_s), to_tensor(y_te, torch.long), task='classification', ...)</code>	Обучаем классификатор.
154	(пустая строка)	Разделитель.
155	<code>pred = predict_classes(model, to_tensor(X_te_s))</code>	Предсказания.
156	<code>acc = classification_metrics(y_te, pred) ['accuracy']</code>	Ассигасу.
157	(пустая строка)	Разделитель.
158	<code>if best_frac is None and acc >= target_acc:</code>	Если ещё не нашли min и ассигасу ≥ 0.90 ...
159	<code>best_frac = float(frac)</code>	...запоминаем frac.
160	(пустая строка)	Разделитель.
161	<code>summary.append({</code>	Добавляем результат.
162	<code>'dataset': name, 'activation': activation,</code>	Датасет и активация.
163	<code>'linear_layers': linear_layers, 'hidden_dim': hidden_dim,</code>	Архитектура.
164	<code>'target_acc': target_acc,</code>	Целевая ассигасу.

Строка	Код	Пояснение
165	<code>'min_frac': best_frac,</code>	Минимальная доля (или <code>None</code> если не найдена).
166	<code>'min_train_size': (None if best_frac is None else int(round(best_frac * len(X)))),</code>	Абсолютное число объектов: $\text{frac} \times n$.
167	<code>})</code>	Конец словаря.
168	(пустая строка)	Разделитель.
169	<code>return summary</code>	Возвращаем список результатов.

5.7. Строки 171–173: `kfold_indices()`

Строка	Код	Пояснение
170	(пустая строка)	Разделитель.
171	<code>def kfold_indices(n, k=5, seed=SEED):</code>	Генерирует k фолдов: списки индексов для кросс-валидации.
172	<code>rng = np.random.default_rng(seed)</code>	Генератор.
173	<code>idx = rng.permutation(n)</code>	Перемешиваем индексы $[0, \dots, n - 1]$.
174	<code>return np.array_split(idx, k)</code>	<code>array_split</code> делит массив на k примерно равных частей. При $n = 500$, $k = 5 \rightarrow 5$ массивов по 100.

5.8. Строки 176–209: `cv_grid_search()`

Строка	Код	Пояснение
175	(пустая строка)	Разделитель.
176	<code>def cv_grid_search(X, y, activations=(...), layers_grid=(2,3,4), hidden_dim=5, k=CV_K, ...):</code>	Grid search с k -fold кросс-валидацией. Перебирает (3 активации $\times$ 3 числа слоёв = 9 комбинаций).
177	<code>X = np.asarray(X, dtype=np.float32)</code>	Приведение типа.
178	<code>y = np.asarray(y, dtype=np.int64)</code>	Приведение типа.
179	<code>folds = kfold_indices(len(X), k=k)</code>	Генерируем k фолдов.
180	(пустая строка)	Разделитель.
181	<code>rows = []</code>	Аккумулятор.
182	<code>best_row = None</code>	Лучшая комбинация.
183	(пустая строка)	Разделитель.
184	<code>for act in activations:</code>	Цикл по активациям.
185	<code>for nl in layers_grid:</code>	Цикл по числу линейных слоёв (2, 3, 4).
186	<code>fold_accs = []</code>	Список ассигасу для каждого фолда.
187	<code>for fi in range(k):</code>	Перебор фолдов: fi — индекс валидационного фолда.
188	<code>val_idx = folds[fi]</code>	Индексы валидации — текущий фолд.
189	<code>tr_idx = np.concatenate([folds[j] for j in range(k) if j != fi])</code>	Индексы обучения — все остальные фолды.
190	(пустая строка)	Разделитель.
191	<code>X_tr, X_val = X[tr_idx], X[val_idx]</code>	Разделение данных.

Строка	Код	Пояснение
192	<code>y_tr, y_val = y[tr_idx], y[val_idx]</code>	Разделение меток.
193	(пустая строка)	Разделитель.
194	<code>X_tr_s, X_val_s, _, _ = standardize(X_tr, X_val)</code>	Стандартизация — заново для каждого фолда.
195	(пустая строка)	Разделитель.
196	<code>model = build_mlp(2, 2, nl-1, hidden_dim, act)</code>	Строим модель. <code>nl-1</code> — скрытых слоёв.
197	<code>train_model(model, to_tensor(X_tr_s), to_tensor(y_tr, torch.long), ...)</code>	Обучаем.
198	(пустая строка)	Разделитель.
199	<code>pred = predict_classes(model, to_tensor(X_val_s))</code>	Предсказания на валидации.
200	<code>fold_accs.append(classification_report(y_val, pred)['accuracy'])</code>	Добавляем ассигуру текущего фолда.
201	(пустая строка)	Разделитель.
202	<code>row = {</code>	Формируем запись.
203	<code>'activation': act, 'linear_layers': nl,</code>	Параметры.
204	<code>'hidden_dim': hidden_dim,</code>	Число нейронов (5).
205	<code>'cv_mean_acc': float(np.mean(fold_accs)),</code>	Среднее ассигуру по k фолдам.
206	<code>'cv_std_acc': (float(np.std(fold_accs, ddof=1)) if len(fold_accs) > 1 else 0.0),</code>	Std с <code>ddof=1</code> — несмещённая оценка (делим на $k - 1$).
207	<code>}</code>	Конец записи.
208	<code>rows.append(row)</code>	Добавляем.
209	<code>if best_row is None or row['cv_mean_acc'] > best_row['cv_mean_acc']:</code>	Обновляем лучшую.
210	<code>best_row = row</code>	Запоминаем.
211	(пустая строка)	Разделитель.
212	<code>return rows, best_row</code>	Возвращаем все результаты и лучшую комбинацию.

5.9. Строки 214–304: `main()`

Строка	Код	Пояснение
213	(пустая строка)	Разделитель.
214	<code>def main():</code>	Точка входа.
215	<code>datasets = load_cls_datasets()</code>	Создаём 4 датасета.
216	(пустая строка)	Разделитель.
217	<code># Визуализация датасетов</code>	Комментарий.
218	<code>fig, axes = plt.subplots(2, 2, figsize=(10, 10))</code>	Сетка 2×2 для обзорного графика.
219	<code>fig.suptitle('Классификационные выборки', fontsize=14, fontweight='bold')</code>	Общий заголовок.

Строка	Код	Пояснение
220	<code>for ax, ds in zip(axes.flat, datasets):</code>	<code>axes.flat</code> — итератор по 4 субплотам; <code>zip</code> связывает <code>subplot</code> ↔ <code>dataset</code> .
221	<code>X, y = ds['data']</code>	Данные.
222	<code>ax.scatter(X[y==0, 0], X[y==0, 1], s=16, alpha=0.75, label='Класс 0')</code>	Точки класса 0.
223	<code>ax.scatter(X[y==1, 0], X[y==1, 1], s=16, alpha=0.75, label='Класс 1')</code>	Точки класса 1.
224	<code>ax.set_title(ds['name'])</code>	Заголовок — имя датасета.
225	<code>ax.legend(fontsize=8)</code>	Легенда.
226	<code>ax.grid(True, alpha=0.3)</code>	Сетка.
227	<code>ax.set_aspect('equal', adjustable='box')</code>	Одинаковый масштаб по осям.
228	<code>plt.tight_layout()</code>	Подгонка.
229	<code>os.makedirs(IMG_DIR, exist_ok=True)</code>	Создание папки.
230	<code>fig.savefig(os.path.join(IMG_DIR, 'cls_datasets.png'), dpi=DPI, bbox_inches='tight')</code>	Сохранение обзорного графика.
231	<code>plt.show()</code>	Показ.
232	<code>plt.close()</code>	Закрытие.
233	(пустая строка)	Разделитель.
234	<code># Перебор архитектур</code>	Комментарий.
235	<code>print(f'\n\nПеребор архитектур (epochs={CLS_EPOCHS}, lr={CLS_LR})...')</code>	Информационное сообщение.
236	<code>rows, best = run_classification_experiments(datasets)</code>	Запуск всех 180 экспериментов ($4 \times 3 \times 15$).
237	(пустая строка)	Разделитель.
238	<code>activations = ['sigmoid', 'tanh', 'relu']</code>	Список для вывода.
239	<code>for ds in datasets:</code>	Для каждого датасета...
240	<code>print('\n\n' + '=' * 70)</code>	Линия.
241	<code>print(ds['name'])</code>	Имя.
242	<code>print('=' * 70)</code>	Линия.
243	<code>for act in activations:</code>	Для каждой активации...
244	<code>print(f'\n\n Активация: {act} (Test accuracy)')</code>	Подзаголовок.
245	<code>print(' ' + '-' * 62)</code>	Линия.
246	<code>print_cls_pivot(rows, ds['name'], act)</code>	Печать таблицы ассурасу.
247	(пустая строка)	Разделитель.
248	<code>print('\n\n\n— Лучшие модели по классификации —')</code>	Заголовок.
249	<code>for (ds_name, act), rec in best.items():</code>	Перебор лучших моделей.
250	<code>print(f' {ds_name} {act}: слоёв=... нейронов=... асс=...')</code>	Вывод.

Строка	Код	Пояснение
251	<code>plot_decision_boundary(rec, save_path=os.path.join(IMG_DIR, f'cls_{ds_name.lower()}_{act}.png'))</code>	Графики + сохранение.
252	(пустая строка)	Разделитель.
253	<code># Минимальный размер выборки</code>	Комментарий.
254	<code>print('\n— Минимальный размер выборки для accuracy >= 90 % —')</code>	Заголовок.
255	<code>size_summary = estimate_min_train_size(datasets)</code>	Запуск поиска.
256	<code>col_w = 14</code>	Ширина столбца.
257	<code>header = (f'{"Датасет":<12}{ "Активация":<{col_w}} { "Слоёв":<{col_w}}" ...)</code>	Формирование заголовка с 7 столбцами.
258	<code>print(header)</code>	Печать.
259	<code>print('-' * len(header))</code>	Линия.
260	<code>for r in size_summary:</code>	Для каждого датасета...
261	<code>mf = f"{r['min_frac']:.2f}" if r['min_frac'] is not None else '-'</code>	Минимальная доля или прочерк.
262	<code>ms = str(r['min_train_size']) if r['min_train_size'] is not None else '-'</code>	Минимальное число объектов или прочерк.
263	<code>print(f"...")</code>	Форматированная строка таблицы.
264	(пустая строка)	Разделитель.
265	<code># Кросс-валидация</code>	Комментарий.
266	<code>print(f'\n— {CV_K}-fold кросс- валидация —')</code>	Заголовок.
267	<code>cv_summary = []</code>	Аккумулятор лучших результатов CV.
268	<code>for ds in datasets:</code>	Для каждого датасета...
269	<code>name = ds['name']</code>	Имя.
270	<code>X, y = ds['data']</code>	Данные.
271	<code>details, best_row = cv_grid_search(X, y)</code>	Запуск grid search с CV.
272	(пустая строка)	Разделитель.
273	<code>cv_summary.append({</code>	Сохраняем лучший результат CV.
274	<code>'dataset': name,</code>	Датасет.
275	<code>'best_activation': best_row['activation'],</code>	Лучшая активация.
276	<code>'best_layers': best_row['linear_layers'],</code>	Лучшее число слоёв.
277	<code>'cv_mean_acc': best_row['cv_mean_acc'],</code>	Среднее ассигасу.
278	<code>'cv_std_acc': best_row['cv_std_acc'],</code>	Std.
279	<code>)</code>	Конец.
280	(пустая строка)	Разделитель.
281	<code>print(f'\n {name}: все комбинации')</code>	Заголовок для датасета.

Строка	Код	Пояснение
282	<pre>print(f" {'Активация':<12} {'Слоёв':<10}{ 'N':<8}{ 'Mean acc':<14} {'Std acc':<14}")</pre>	Заголовок таблицы.
283	<pre>print(' ' + '-' * 56)</pre>	Линия.
284	<pre>for r in sorted(details, key=lambda x: x['cv_mean_acc']):</pre>	Сортировка по mean acc (от худшего к лучшему).
285	<pre>print(f" {r['activation']:<12} {r['linear_layers']:<10} {r['hidden_dim']:<8} {r['cv_mean_acc']:<14.4f} {r['cv_std_acc']:<14.4f}")</pre>	Печать строки.
286	(пустая строка)	Разделитель.
287	<pre>print(f'\n\n Лучшие результаты {CV_K}- fold CV:')</pre>	Заголовок итоговой таблицы.
288	<pre>print(f" {'Датасет':<12} {'Активация':<12}{ 'Слоёв':<10}{ 'Mean acc':<14}{ 'Std acc':<14}")</pre>	Заголовок.
289	<pre>print(' ' + '-' * 60)</pre>	Линия.
290	<pre>for r in cv_summary:</pre>	Для каждого датасета...
291	<pre>print(f" {r['dataset']:<12} {r['best_activation']:<12} {r['best_layers']:<10} {r['cv_mean_acc']:<14.4f} {r['cv_std_acc']:<14.4f}")</pre>	Итоговая строка: лучшая комбинация для каждого датасета.
292	(пустая строка)	Разделитель.
293	(пустая строка)	Дополнительный разделитель.
294	<pre>if __name__ == '__main__':</pre>	Проверка: файл запущен как скрипт?
295	<pre>main()</pre>	Вызов main().